

Day 40

* Parameters and Arguments :-

Parameter: The Variable used in function Heading are called "Formal parameters" and they are used for storing the inputs coming function calls.

→ The Variables used in function Body are called "Local Variables/ parameters and they are used for storing temporary results/ function processing logic results.

→ The Values formal parameters and local parameters can be accessed within corresponding function definition but not possible to access in other part of the program.

Examples:

```
def sumop(a,b): # Here a,b are called formal parameters.
    c = a + b
```

Arguments: - Arguments are the variables/values which are used as variables in function calls.

Examples:

```
sumop(10,20) # Here 10, 20 are called 'Argument value'
(OR)
a = 10
b = 20
```

```
sumop(a,b) # Here a,b are called Arguments (Actual variables)
```

→ The relationship between Arguments and parameters is that all the values of arguments are passing to parameters. This mechanism is called "Argument passing".

* Types of Arguments and parameters :-

→ As we know that all the values of arguments are passing to parameters.

→ Based on passing the values of Arguments from Function to formal parameters of Function.

Definition: the Arguments are classified into 5 Types. They are:

- ① positional Arguments.
- ② Default Arguments.
- ③ keyword Arguments.
- ④ variable length Arguments
- ⑤ keyword variables length Arguments

① positional Arguments :- (or) parameters

→ The concept of positional Arguments (or) parameters says that "The number of Arguments of function call must be equal to the number of formal parameters in Function Heading".

→ This parameter mechanism also recommends to follow order and meaning of parameters for high accuracy.

→ To pass the specific Data from Function call to Function definition then we must take positional Argument mechanism.

→ The default Argument passing mechanism is positional Arguments or parameters.

Syntax for function definition:-

```
def functionname (Param1, Param2, ... Param-n):
    _ _ _ _ _
```

Syntax for function call:-

```
functionname (arg1, arg2, ... arg-n)
```

→ Here the values of arg1, arg2, ... arg-n are passing to Param-1, Param-2, ... Param-n respectively.

Rule: when we use default parameters in the function definition, they must be used as last parameter(s) otherwise we get key error (syntax error:- non default arguments (positional) follows default arguments)

program for demonstrating the concept of default Arguments:

ex 1. `def dispstudinfo(sno, sname, marks, crs="python"):`
`print("\t{sno}\t{sname}\t{marks}\t{crs}".format(sno, sname, marks, crs)).`

main program.

`print("=" * 50)`

`print("\t{sno}\t{sname}\t{marks}\t{crs}")`

`print("=" * 50)`

`dispstudinfo(100, "Raj", 34.56)`
`dispstudinfo(200, "Raj", 44.55)`
`dispstudinfo(300, "Raj", 14.55)`
`print("=" * 50)`

} # function call

ex 2. `def` → same `def`

main prog:- `500, trump, 10.11`
`add Hoge`. `600, Biden, 30.11, "politics"` (automatic `crs` `python` show)
`700, vivek, 40.11`

ex 3. `def dispstudinfo(sno, sname, marks, crs="python", cnt="india"):-`

main prog.

`add Hoge`.

`500, trump, 10.11, cnt=usa`

`600, Biden, 30.11, politics, usa`

`700, vivek, 40.11`

ex 4. `def` ← `dispstudinfo 1` (python waale) — `Raj, Trump, Biden, vivek`,
`def` ← `dispstudinfo 2` (Java waale) — `Trump, Biden, vivek`.

ex 5.


```

(5) . def circlearea_peri (r, PI=3.14):
    ac = PI * r ** 2
    pc = 2 * PI * r
    print ("radius: {}".format(r))
    print ("Area of circle: {}".format(ac))
    print ("Peri of circle: {}".format(pc))

# main program
circlearea_peri (float(input("Enter radius:")))

```

(3) Keyword parameters (or) arguments:-

→ In some of the circumstances, we know the function name and formal parameter names and we don't know the order of formal parameter names and to pass the data/values accurately we must use the concept of keyword parameters (or) arguments.

→ the implementation of keyword parameters (or) arguments says that all the formal parameter names used as arguments in function call(s) as keys.

syntax for function definition:-

```
def _functionname_ (param1, param2, ... param-n):
```

syntax for function call:-

```
functionname (param-n=val-n, param1=val1,
              param-n-1=val-n-1, . . . . .)
```

→ Here param-n=val-n, param1=val1, param-n-1=val-n-1, are called keyword arguments.

→ when we specify keyword arguments before positional arguments in function call(s) then we get syntax error: positional argument follows keyword argument.

program for demonstrating keyword arguments:-

```
def disp(a,b,c,d):
    print("\t {} \t {} \t {} \t {}".format(a,b,c,d))
```

main program

```
print("=" * 50)
```

```
print("\t A \t B \t C \t D")
```

```
print("=" * 50)
```

```
disp(10,20,30,40) # Function call with positional args.
```

```
disp(d=40, a=10, c=30, b=20) # with keywords args.
```

```
disp(b=20, c=30, d=40, a=10) # with keywords args.
```

```
disp(10,20, d=40, c=30) # with positional args & keyword args.
```

```
# disp(d=40, c=30, 10, 20) # syntax error.
```

```
disp(10, d=40, b=20, c=30) # with positional args & keyword args
```

```
print("=" * 50)
```

python wala example

ex 12

```
# main dispstudinfo(100, "monica", 66.66) # with poss args.
```

```
dispstudinfo(marks=65.66, sno=200, # with keywords.
              sname="sushma")
```

```
dispstudinfo(crs="java", sno=300, sname="sangita", marks=55.55)
# with keywords.
```

```
dispstudinfo(sname="prajwala", marks=66.77, crs="bsc", sno=400)
# with keyword
```

```
dispstudinfo(500, "vanya", crs="java", marks=22.22) # Call with
# positional args & keyword args
```

Day (45):-

④ Variable length parameters (or) ~~multiple~~ arguments

→ when we have family of multiple function calls with variable number of values/arguments then with normal python programming, we must define multiple function definitions.

This process leads to more development time.

→ To overcome this process, we must use the concept of variable length parameters.

→ To implement variable length parameters concept, we must define single function definition and takes a formal parameter preceded with a symbol called asterisk (*param). and the formal parameter with asterisk symbol is called variable length parameters and whose purpose is to hold/store any number of values coming from similar function calls and whose type is <class, 'tuple'>

Syntax for function definition with variable length parameters:-

```
def functionname (list of formal params, *param1,
                  param2=value):
```

⇒ Here *param1 is called variable length parameters and it can hold any number of argument values (or) variable number of argument values and *param1 type is <class, 'tuple'>

⇒ Rule 1:- the *param1 must always written at last part of function heading and it must be only one (but not multiple)

⇒ Rule 2:- when we use variable length and default parameters in function heading, we use default parameter as last and before we use variable length parameter and in function calls, we should not use default parameter as keyword argument bcoz variable number of values are treated as positional argument value(s).

program for demonstrate variable arguments.

```
def disp(a,b,c,d): # function def-①
    print(a,b,c,d)
```

```
def disp(a,b,c): # function def-②
    print(a,b,c)
```

ex ①

```
def disp(a,b): # function def-③
    print(a,b)
```

```
def disp(a): # function def-④
    print(a)
```

main program

```
disp(10,20,30,40) # function call 1
```

```
disp(10,20,30) # function call 2
```

```
disp(10,20) # function call ③
```

```
disp(10) # function call ④
```

This prog. will not execute as it is

ex ②

```
def disp(a,b,c,d): # function def-1
    print(a,b,c,d)
```

```
disp(10,20,30,40) # function call-①
```

```
def disp(a,b,c): # function def ②
    print(a,b,c)
```

```
disp(10,20,30) # function call ②
```

```
def disp(a,b):
```

```
    print(a,b)
```

```
disp(10,20)
```

```
def disp(a):
```

```
    print(a)
```

```
disp(10)
```

=.

Pure Variable

Ex 10

```
def disp(*k): # *param is called Variable length
              # parameter - whose type is tuple.
    print("-" * 50)
    print("Number of values: {}".format(len(k)))
    for v in k:
        print("{} ".format(v), end=" ")
    print()
    print("-" * 50)
```

#main program

```
disp(10, 20, 30, 40) # function call ①
disp(10, 20, 30)     # ②
disp(10, 20)         # ③
disp(10)              # ④
disp()                # ⑤
disp(10, "Rakesh", 45-67, True, 2+3j)
```

Ex 11

```
def findsum(sno, sname, *vals):
    print("-" * 50)
    print("Student ID: {}".format(sno))
    print("Student Name: {}".format(sname))
    s = 0
    for val in vals:
        print("{} ".format(val))
        s = s + val
    print("-" * 50)
    print("sum = {}".format(s))
    print("-" * 50)
```

#main program

```
findsum(100, "Roshan", 10, 20, 30, 40, 50) # function call ①
findsum(200, "Ravi", 10, 20, 30, 40)       # ②
findsum(300, "Kinney", 10, 20, 20)         # ③
findsum(400, "Dennis", 10, 20)            # ④
findsum(500, "Nif", 10)                    # ⑤
findsum(600, "KUR")                        # ⑥
```

Ex ③. def = *val, city = "HYD"
 using city. Baaki sab samu

Ex ④. #main program.
findsum(400, "Dennis", city = "RSP", 10, 20) # function call
findsum(400, Dennis, 10, 20, city = "RSP") — syntax error
city = "WH"
Baaki sab samu

⑤. keyword variable length parameters (or) argument:-

→ When we have family of multiple function calls with keyword variable number of values/arguments then with normal Python programming, we must define multiple function definitions. This process leads to more development time.

→ To overcome this process, we must use the concept of keyword variable length parameters.

→ To implement keyword variable length parameters concept, we must define single function definition and takes a formal parameter preceded with a symbol called double asterisk (**param). and the formal parameter with double asterisk symbol is called keyword variable length parameters and whose purpose is to hold/store any number of (key, value) coming from similar function calls and whose type is <class, 'dict'>

syntax for function definition with keyword variable length param

def functionname (list of formal param, **param):

=> Here `**param` is called keyword variable length parameter and it can hold any number of keyword argument values (or) keyword variable number of argument values and `**param` type is <class, 'dict'>

=> Rule:- the `**param` must always written at last part of function heading and it must be only one (but not multiple).

Final syntax for defining a function

```
def funcname(pos formal params, *valenparams,
              default params, **kwvarlenparams):
```

program for demonstrating keyword variable length Arguments

X def 'disp (a,b,c,d):- function def ①.

```
    print (" - - - ")
    print (a,b,c,d)
    print (" - - - ")
```

①

```
def disp(x,y,z):
    print(x,y,z)
def disp(k,l,m,n)
    print(k,l,m,n)
```

~~def disp~~ main program -

```
disp (c=30, d=40, a=10, b=20)
disp (X=HYD, Y=Bang, z=Mum)
disp (K=100, L=bassum, m=Python, N=33.33)
```

ex ② def ke niche hi
disp (likkr print kr degye)


```
b=20
sumop(a,b)#here a,b are called arguments actual vaariables
```

Types of Arguments and parameters:

- 1. positional Arguments
- 2. Default Arguments
- 3. Keyword Arguments
- 4. variable Arguments
- 5. keyword variables length arguments

1. possitional arguments:

In [6]: *#program for demonstrating the concept pf possitional Arguments:*

```
def dispstudinfo(sno,sname,marks):
    print("\t{} \t{} \t{}".format(sno,sname,marks))
#main program
print("="*50)
print("\t sno\t sname \t marks:")
print("="*50)
dispstudinfo(100,"rs",34.56)
dispstudinfo(200,"tr",44.55)
dispstudinfo(300,"dr",14.55)
dispstudinfo(400,"kv",11.11)
print("="*50)
```

```
=====
          sno      sname   marks:
=====
          100      rs      34.56
          200      tr      44.55
          300      dr      14.55
          400      kv      11.11
=====
```

In [7]: **def** dispstudinfo(sno,sname,marks,crs):
 print("\t{} \t{} \t{} \t{}".format(sno,sname,marks,crs))

```
#main program
print("="*50)
print("\tsno\tsname\tmarks\tcrs:")
print("="*50)
dispstudinfo(100,"rs",34.56,"python")
dispstudinfo(200,"tr",44.55,"python")
dispstudinfo(300,"dr",14.55,"python")
dispstudinfo(400,"kv",11.11,"python")
print("="*50)
```

```
=====
          sno      sname   marks   crs:
=====
          100      rs      34.56   python
          200      tr      44.55   python
          300      dr      14.55   python
          400      kv      11.11   python
=====
```

2. Default parameters or Arguments

```
In [9]: #program for demonstrating the concept of default parameters
def dispstudinfo(sno,sname,marks,crs="python"):
    print("\t{}\t{}\t{}\t{}".format(sno,sname,marks,crs))
#main program
print("="*50)
print("\tsno\t\sname\t\marks\t\course")
print("="*50)
dispstudinfo(100,"rs",34.56)
dispstudinfo(200,"tr",44.55)
dispstudinfo(300,"dr",14.55)
dispstudinfo(400,"kv",11.11)
print("="*50)
```

```
=====
sno      sname   marks   course
=====
100      rs       34.56   python
200      tr       44.55   python
300      dr       14.55   python
400      kv       11.11   python
=====
```

```
In [10]: #program for demonstrating the concept of default parameters
def dispstudinfo(sno,sname,marks,crs="python"):
    print("\t{}\t{}\t{}\t{}".format(sno,sname,marks,crs))
#main program
print("="*50)
print("\tsno\t\sname\t\marks\t\course")
print("="*50)
dispstudinfo(100,"rs",34.56)
dispstudinfo(200,"tr",44.55)
dispstudinfo(300,"dr",14.55)
dispstudinfo(400,"kv",11.11)
dispstudinfo(500,"trump",10.11)
dispstudinfo(600,"biden",30.11,"politics")
dispstudinfo(700,"vivek",40.11)
print("="*50)
```

```
=====
sno      sname   marks   course
=====
100      rs       34.56   python
200      tr       44.55   python
300      dr       14.55   python
400      kv       11.11   python
500      trump    10.11   python
600      biden    30.11   politics
700      vivek    40.11   python
=====
```

```
In [12]: def dispstudinfo(sno,sname,marks,crs="python",cnt="india"):
    print("\t{}\t{}\t{}\t{}\t{}".format(sno,sname,marks,crs,cnt))
#main program
#main program
print("="*50)
print("\tsno\t\sname\t\marks\t\course\t\country")
print("="*50)
dispstudinfo(100,"rs",34.56)
dispstudinfo(200,"tr",44.55)
dispstudinfo(300,"dr",14.55)
```

```
dispstudinfo(400,"kv",11.11)
dispstudinfo(500,"trump",10.11,cnt="USA")
dispstudinfo(600,"biden",30.11,"politics","USA")
dispstudinfo(700,"vivek",40.11)
print("="*50)
```

```
=====
      sno      sname   marks   course   country
=====
      100       rs     34.56   python   india
      200       tr     44.55   python   india
      300       dr     14.55   python   india
      400       kv     11.11   python   india
      500      trump    10.11   python   USA
      600      biden    30.11   politics  USA
      700      vivek    40.11   python   india
=====
```

```
In [16]: def dispstudinfo1(sno,sname,marks,crs="python"):
          print("\t{}\t{}\t{}\t{}".format(sno,sname,marks,crs))
          def dispstudinfo2(sno,sname,marks,crs="java"):
              print("\t{}\t{}\t{}\t{}".format(sno,sname,marks,crs))
          #main program
          print("="*50)
          print("\tsno\tsname\tmarks\tcourse")
          print("="*50)
          dispstudinfo1(100,"rs",34.56)
          dispstudinfo1(200,"tr",44.55)
          dispstudinfo1(300,"dr",14.55)
          dispstudinfo1(400,"kv",11.11)
          dispstudinfo2(500,"trump",10.11)
          dispstudinfo2(600,"biden",30.11)
          dispstudinfo2(700,"vivek",40.11)
          print("="*50)
```

```
=====
      sno      sname   marks   course
=====
      100       rs     34.56   python
      200       tr     44.55   python
      300       dr     14.55   python
      400       kv     11.11   python
      500      trump    10.11   java
      600      biden    30.11   java
      700      vivek    40.11   java
=====
```

```
In [19]: def circleareaper(r,pi=3.14):
          ac=pi*r**2
          pc=2*pi*r
          print("radius:{}".format(r))
          print("Area of circle:{}".format(ac))
          print("perimeter of circle:{}".format(pc))
          #main program
          circleareaper(float(input("enter radius")))
```

```
radius:5.0
Area of circle:78.5
perimeter of circle:31.400000000000002
```

3.keyword parameters or arguments:

In [23]: *#program for demonstrating keyword arguments:*

```
def disp(a,b,c,d):
    print("\t{}\t{}\t{}\t{}".format(a,b,c,d))
#main program
print("="*50)
print("\tA\tB\tC\tD")
print("="*50)
disp(10,20,30,40)
disp(d=40,a=10,b=20,c=30)
disp(b=20,c=50,d=10,a=40)
disp(10,20,c=50,d=120)
disp(10,20,30,d=49)
disp(10,d=40,b=20,c=30)
print("="*50)
```

```
=====
      A      B      C      D
=====
      10      20      30      40
      10      20      30      40
      40      20      50      10
      10      20      50      120
      10      20      30      49
      10      20      30      40
=====
```

In [31]: *def dispstudinfo1(sno,sname,marks,crs="python"):*

```
    print("\t{}\t{}\t\t\t{}\t{}".format(sno,sname,marks,crs))
#main program
print("="*50)
print("\tsno\t\sname\t\t marks\t\course")
print("="*50)
dispstudinfo(100,"monica",66.66)
dispstudinfo(marks=65.66,sno=200,sname="sushma")
dispstudinfo(crs="java",sno=300,sname="sangita",marks=55.55)
dispstudinfo(sname="prajawal",marks=66.77,crs="bsc",sno=400)
dispstudinfo(500,"uday",crs="java",marks=22.22)
```

```
=====
      sno      sname      marks      course
=====
      100      monica  66.66      python  india
      200      sushma  65.66      python  india
      300      sangita  55.55      java    india
      400      prajawal          66.77      bsc     india
      500      uday    22.22      java    india
```

4.Variable length parameters or arguments:

In [33]: *def disp(*k):*

```
    print("-"*50)
    print("number of values:{}".format(len(k)))
    for v in k:
        print("{} ".format(v),end=" ")
    print()
    print("-"*50)
#main program
disp(10,20,30,40)
```

```

disp(10,20,30)
disp(10,20)
disp(10)
disp()
disp(10,"rakesh",45.67,True,2+3j)

```

```

-----
number of values:4

```

```

10 20 30 40

```

```

-----
number of values:3

```

```

10 20 30

```

```

-----
number of values:2

```

```

10 20

```

```

-----
number of values:1

```

```

10

```

```

-----
number of values:0

```

```

-----
number of values:5

```

```

10 rakesh 45.67 True (2+3j)
-----

```

```

In [39]: def findsum(sno,sname,*vals):
          print("="*50)
          print("\tstudent id:{}".format(sno))
          print("\tstudent Name:{}".format(sname))
          s=0
          for val in vals:
              print("\t{}:".format(val))
              s=s+val
          print("="*50)
          print("sum={}".format(s))
          print("="*50)
          #main program
          findsum(100,"rosum",10,20,30,40,50)
          findsum(200,"travis",10,20,30,40)
          findsum(300,"kinney",10,20,30)
          findsum(400,"dennis",10,20)
          findsum(500,"arif",10)
          findsum(600,"babu")

```

```

=====
        student id:100
        student Name:rosum
        10:
        20:
        30:
        40:
        50:
=====
sum=150
=====
=====
        student id:200
        student Name:travis
        10:
        20:
        30:
        40:
=====
sum=100
=====
=====
        student id:300
        student Name:kinney
        10:
        20:
        30:
=====
sum=60
=====
=====
        student id:400
        student Name:dennis
        10:
        20:
=====
sum=30
=====
=====
        student id:500
        student Name:arif
        10:
=====
sum=10
=====
=====
        student id:600
        student Name:babu
=====
sum=0
=====

```

```

In [42]: def findsum(sno,sname,*vals,city="hyd"):
        print("="*50)
        print("\tstudent id:{}".format(sno))
        print("\tstudent Name:{}".format(sname))
        print("\tLiving city:{}".format(city))
        s=0
        for val in vals:
            print("\t{}:".format(val))

```

```
        s=s+val
    print("="*50)
    print("sum={}".format(s))
    print("="*50)
#main program
findsum(100,"rosum",10,20,30,40,50)
findsum(200,"travis",10,20,30,40)
findsum(300,"kinney",10,20,30)
findsum(400,"dennis",10,20)
findsum(500,"arif",10)
findsum(600,"babu")
```



```

=====
        student id:100
        student Name:rosum
        Living city:hyd
        10:
        20:
        30:
        40:
        50:
=====
sum=150
=====
=====
        student id:200
        student Name:travis
        Living city:hyd
        10:
        20:
        30:
        40:
=====
sum=100
=====
=====
        student id:300
        student Name:kinney
        Living city:hyd
        10:
        20:
        30:
=====
sum=60
=====
=====
        student id:400
        student Name:dennis
        Living city:hyd
        10:
        20:
=====
sum=30
=====
=====
        student id:500
        student Name:arif
        Living city:hyd
        10:
=====
sum=10
=====
=====
        student id:600
        student Name:babu
        Living city:hyd
=====
sum=0
=====

```

```

In [43]: def findsum(sno,sname,*vals,city="hyd"):
          print(""*50)

```

```
print("\tstudent id:{} ".format(sno))
print("\tstudent Name:{} ".format(sname))
print("\tLiving city:{} ".format(city))
s=0
for val in vals:
    print("\t{}:".format(val))
    s=s+val
print("="*50)
print("sum={} ".format(s))
print("="*50)
#main program
findsum(100,"rosum",10,20,30,40,50)
findsum(200,"travis",10,20,30,40)
findsum(300,"kinney",10,20,30)
findsum(400,"dennis",10,20)
findsum(500,"arif",10,city="RSA")
findsum(600,"babu",city="usa")
```

```
=====
    student id:100
    student Name:rosum
    Living city:hyd
    10:
    20:
    30:
    40:
    50:
=====
sum=150
=====
=====
    student id:200
    student Name:travis
    Living city:hyd
    10:
    20:
    30:
    40:
=====
sum=100
=====
=====
    student id:300
    student Name:kinney
    Living city:hyd
    10:
    20:
    30:
=====
sum=60
=====
=====
    student id:400
    student Name:dennis
    Living city:hyd
    10:
    20:
=====
sum=30
=====
=====
    student id:500
    student Name:arif
    Living city:RSA
    10:
=====
sum=10
=====
=====
    student id:600
    student Name:babu
    Living city:usa
=====
sum=0
=====
```

5.keyword variables length arguments

- syntax:- def functionname(list of formal param.**param):

```
In [3]: #program for demonstrating keyword variable length arguments
#this program will not execute
def disp (a,b,c,d):
    print("=====")
    print(a,b,c,d)
    print("=====")
def disp (x,y,z):
    print(x,y,z)
def disp(k,l,m,n):
    print(k,l,m,n)
#main program
#disp(c=30,d=40,a=10,b=20)
disp(x="hyd",y="bang",z="mum")
disp(k=100,l="rosum",m="python",n=33.33)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[3], line 13
     10     print(k,l,m,n)
     11 #main program
     12 #disp(c=30,d=40,a=10,b=20)
--> 13 disp(x="hyd",y="bang",z="mum")
     14 disp(k=100,l="rosum",m="python",n=33.33)

TypeError: disp() got an unexpected keyword argument 'x'
```

```
In [4]: #Program for demonstrating Keyword Variable Length Arguments
#Ex2.py
def disp(a,b,c,d): #Function Def-1
    print("-----")
    print(a,b,c,d)
    print("-----")

disp(c=30,d=40,a=10,b=20) # Function call-1

def disp(x,y,z): #Function Def-2
    print("-----")
    print(x,y,z)
    print("-----")

disp(x="HYD",y="BANG",z="MUM") # Function call-2

def disp(K,L,M,N): #Function Def-3
    print("-----")
    print(K,L,M,N)
    print("-----")

disp(K=100,L="ROSSUM",M="PYTHON",N=33.33) # Function call-3
```



```
-----
10 20 30 40
-----
-----
```

```
HYD BANG MUM
-----
-----
```

```
100 ROSSUM PYTHON 33.33
-----
```

```
In [5]: #Program for demonstrating Keyword Variable Length Arguments
#PureKwdVarLenArgsEx1.py
def disp( ** k): # here **k is called variable Keyword Length args--whose type
    print("-----")
    print(k,type(k))
    print("-----")

#main program
disp(c=30,d=40,a=10,b=20) # Function call-1
disp(city1="HYD",city2="BANG",city3="MUM") # Function call-2
disp(id=100,name="ROSSUM",sub="PYTHON",marks=33.33) # Function call-3
```

```
-----
{'c': 30, 'd': 40, 'a': 10, 'b': 20} <class 'dict'>
-----
-----
```

```
-----
{'city1': 'HYD', 'city2': 'BANG', 'city3': 'MUM'} <class 'dict'>
-----
-----
```

```
-----
{'id': 100, 'name': 'ROSSUM', 'sub': 'PYTHON', 'marks': 33.33} <class 'dict'>
-----
```

```
In [6]: #Program for demonstrating Keyword Variable Length Arguments
#PureKwdVarLenArgsEx2.py
def disp( ** k): # here **k is called variable Keyword Length args--whose type
    print("-----")
    for kv,vv in k.items():
        print("\t{ }-->{ }".format(kv,vv))
    print("-----")

#main program
disp(c=30,d=40,a=10,b=20) # Function call-1
disp(city1="HYD",city2="BANG",city3="MUM") # Function call-2
disp(id=100,name="ROSSUM",sub="PYTHON",marks=33.33) # Function call-3
disp(sid=123, sname="Mahesh",hobby1="Eating",hobby2="Sleep",hobby3="Cahtting")
```

```

-----
c-->30
d-->40
a-->10
b-->20
-----

```

```

-----
city1-->HYD
city2-->BANG
city3-->MUM
-----

```

```

-----
id-->100
name-->ROSSUM
sub-->PYTHON
marks-->33.33
-----

```

```

-----
sid-->123
sname-->Mahesh
hobby1-->Eating
hobby2-->Sleep
hobby3-->Cahtting
-----

```

```

In [7]: #Program for demonstrating Keyword Variable Length Arguments
#PureKwdVarLenArgsEx3.py
def findtotalmarks(sno,sname,cls,**submarks):
    print("-----")
    print("\tStudent Number:{}".format(sno))
    print("\tStudent Name:{}".format(sname))
    print("\tStudent Class:{}".format(cls))
    print("-----")
    tm=0
    print("\tSubject\t\tMarks")
    print("-----")
    for sn,sm in submarks.items():
        print(" \t{}\t\t{}".format(sn,sm))
        tm=tm+sm
    print("-----")
    print("\tTOTAL MARKS:{}".format(tm))
    print("-----")

#main program
findtotalmarks(100,"Rajesh","X",Tel=60,Hindi=70,English=67,Maths=60,Science=60,S
findtotalmarks(200,"Navenn","XII",Maths=73,Physics=58,Chemistry=60)
findtotalmarks(400,"Veersh","BE",OS=60,DBMS=75,CG=55,SET=62)
findtotalmarks(500,"Rossum","Reserch")

```

Student Number:100	
Student Name:Rajesh	
Student Class:X	

Subject	Marks

Tel	60
Hindi	70
English	67
Maths	60
Science	60
Social	75

TOTAL MARKS:392	

Student Number:200	
Student Name:Navenn	
Student Class:XII	

Subject	Marks

Maths	73
Physics	58
Chemistry	60

TOTAL MARKS:191	

Student Number:400	
Student Name:Veersh	
Student Class:BE	

Subject	Marks

OS	60
DBMS	75
CG	55
SET	62

TOTAL MARKS:252	

Student Number:500	
Student Name:Rossum	
Student Class:Reserch	

Subject	Marks

TOTAL MARKS:0	

```
In [8]: #Program for demonstrating Keyword Variable Length Arguments
#PureKwdVarLenArgsEx4.py
def findtotalmarks(sno,sname,cls,state="TS",**submarks):
    print("-----")
    print("\tStudent Number:{}".format(sno))
    print("\tStudent Name:{}".format(sname))
```

```

print("\tStudent Class:{} ".format(cls))
print("\tStudent State:{} ".format(state))
print("-----")
tm=0
print("\tSubject\t\tMarks")
print("-----")
for sn,sm in submarks.items():
    print(" \t{}\t\t{}".format(sn,sm))
    tm=tm+sm
print("-----")
print("\tTOTAL MARKS:{} ".format(tm))
print("-----")
#main program
findtotalmarks(100,"Rajesh","X",Tel=60,Hindi=70,English=67,Maths=60,Science=60,S
findtotalmarks(200,"Navenn","XII",Maths=73,Physics=58,Chemistry=60)
findtotalmarks(400,"Veersh","BE",OS=60,DBMS=75,CG=55,SET=62,state="MUM")
findtotalmarks(500,"Rossum","Reserch",state="NL")

```

Student Number:100	
Student Name:Rajesh	
Student Class:X	
Student State:TS	
Subject	Marks
Tel	60
Hindi	70
English	67
Maths	60
Science	60
Social	75
TOTAL MARKS:392	
Student Number:200	
Student Name:Navenn	
Student Class:XII	
Student State:TS	
Subject	Marks
Maths	73
Physics	58
Chemistry	60
TOTAL MARKS:191	
Student Number:400	
Student Name:Veersh	
Student Class:BE	
Student State:MUM	
Subject	Marks
OS	60
DBMS	75
CG	55
SET	62
TOTAL MARKS:252	
Student Number:500	
Student Name:Rossum	
Student Class:Reserch	
Student State:NL	
Subject	Marks
TOTAL MARKS:0	

In [9]: *#Program for demonstrating Keyword Variable Length Arguments*
#PureKwdVarLenArgsEx5.py


```

def findtotalmarks(sno,sname,cls,*vals,state="TS",**submarks):
    print("=====")
    print("Variable Length Values")
    for val in vals:
        print("\t\t{}".format(val))
    print("=====")
    print("-----")
    print("\tStudent Number:{}".format(sno))
    print("\tStudent Name:{}".format(sname))
    print("\tStudent Class:{}".format(cls))
    print("\tStudent State:{}".format(state))
    print("-----")
    tm=0
    print("\tSubject\t\tMarks")
    print("-----")
    for sn,sm in submarks.items():
        print(" \t{}\t\t{}".format(sn,sm))
        tm=tm+sm
    print("-----")
    print("\tTOTAL MARKS:{}".format(tm))
    print("-----")

#main program
findtotalmarks(100,"Rajesh","X",1,2,3,4,5,Tel=60,Hindi=70,English=67,Maths=60,Sc
findtotalmarks(200,"Navenn","XII",10,20,30,40,Maths=73,Physics=58,Chemistry=60)
findtotalmarks(400,"Veersh","BE",100,200,OS=60,DBMS=75,CG=55,SET=62,state="MUM")
findtotalmarks(500,"Rossum","Reserch",-1,-2,-3,-4,-5,state="NL")
findtotalmarks(600,"KVR","Trainer")

```

```
=====
Variable Length Values
    1
    2
    3
    4
    5
=====

-----
Student Number:100
Student Name:Rajesh
Student Class:X
Student State:TS
-----

Subject      Marks
-----
Tel          60
Hindi        70
English      67
Maths        60
Science      60
Social       75
-----

TOTAL MARKS:392
-----

=====
Variable Length Values
    10
    20
    30
    40
=====

-----
Student Number:200
Student Name:Navenn
Student Class:XII
Student State:TS
-----

Subject      Marks
-----
Maths        73
Physics      58
Chemistry    60
-----

TOTAL MARKS:191
-----

=====
Variable Length Values
    100
    200
=====

-----
Student Number:400
Student Name:Veersh
Student Class:BE
Student State:MUM
-----

Subject      Marks
-----
OS           60
-----
```

DBMS	75
CG	55
SET	62

TOTAL MARKS:252

=====

Variable Length Values
-1
-2
-3
-4
-5

=====

Student Number:500
Student Name:Rossum
Student Class:Reserch
Student State:NL

Subject	Marks
---------	-------

TOTAL MARKS:0

=====

Variable Length Values

=====

Student Number:600
Student Name:KVR
Student Class:Trainer
Student State:TS

Subject	Marks
---------	-------

TOTAL MARKS:0

In []: